ZECPATH AI - Internship Portfolio

Project summary, engineering evidence, and responsible-use scope

Portfolio profile

Author: Asif C
Project: ZECPATH AI
Focus: Modular Python prototype for AI-assisted recruitment workflows

Project summary

ZECPATH AI explores how software components can help organize hiring work: reading resume and job-description documents, comparing job-related evidence, supporting screening and interviews, combining scores, and recording decision-support results. The repository emphasizes configurable modules, synthetic end-to-end demonstrations, automated tests, and documentation of limits.

This is a local prototype and demonstration environment. It is not a deployed hiring service and does not replace recruiter judgment.

Problem and approach

Recruitment can involve unstructured documents, repeated interview stages, and scores that are hard to compare. The project separates these tasks into callable modules with explicit inputs and outputs. The Day 56 and Day 63 scripts show selected handoffs using fictional data; most modules can also be called independently.

Prototype flow

Resume and job-description documents > parsing and structured records > candidate-to-job ATS score > screening and interview components > unified score and eligibility checks > explainable recommendation for human review.

The arrows describe available components and demonstration paths. Resume parsing is not yet automatically connected to ATS matching, and the repository does not expose an implemented HTTP API or connected hiring application.

Main modules

- **Parsing:** PDF/DOCX readers, resume text/section parsing, job-description parsing, and schema normalization.
- **Scoring:** semantic ATS matching, screening aggregation, five-round unified scoring, eligibility rules, explainability, and recommendation generation.
- **Screening and interviews:** conversation state, response analysis, HR interview flow, technical question flow, and rubric-based machine-test evaluation.
- **Governance helpers:** consent records, access-policy checks, audit events, fairness analysis, and retention decisions.

Engineering Work and Evidence

Selected repository capabilities and completed validation

Engineering practices represented

- Python packages separate parsing, scoring, screening, interview, compliance-support, and utility responsibilities.
- JSON rules make score weights, thresholds, interview behavior, and policy settings inspectable and configurable.
- Schema adapters and finite-score validation handle alternate input structures and edge values such as missing, malformed, or non-finite scores.
- Atomic JSON writing, score explanations, and audit events support reproducible and reviewable outputs.
- Pytest covers scoring, data contracts, conversation flows, edge cases, explainability, fairness, consent, and retention helpers.

Milestone evidence

- **Day 55-62:** security/governance materials, performance and observability plans, and architecture/module references document the design and current boundaries.
- **Day 56:** synthetic flow joins selected ATS, screening, HR, technical, unified-scoring, and decision components. It uses fixtures and is not a production intake path.
- **Day 63:** a manifest organizes two fictional job descriptions, four separate candidate profiles, and a shared assessment fixture for an eight-stage demo.
- **Day 66-67:** demo presentation materials and rehearsal notes explain the synthetic flow, timing, sample results, and stakeholder questions.
- **Day 68:** isolated a test fixture side effect and completed the final local validation run.

Day 68 validation snapshot

Automated suite: 679 passed and 8 subtests passed.
Compilation: 8 source/test folders and 18 root entry points compiled successfully.
End-to-end check: four synthetic candidates, all expected stages present, scores within 0-100, and unified weighted contributions matching reported totals.
Timing: the Day 63 run took about 1.02 seconds after model initialization; the separate eight-resume warm benchmark averaged about 0.128 seconds per resume in that local run.

Measurements are single local runs, not a load test, service-level objective, or proof of hiring accuracy.

Day 69 handoff artifacts

- Developer setup and run instructions: repository root README.
- AI roadmap and editable portfolio source: docs/day69/.

Responsible Use and Next Steps

Current limitations and the path toward a governed pilot

What the demonstration does not establish

- Day 63 reads structured fictional resume JSON; it does not exercise a production upload endpoint or parser-to-ATS integration.
- HR and technical stages use deterministic or rubric-based inputs. The machine-test evaluator scores supplied fixtures and does not execute candidate code in a sandbox.
- Consent, access, audit, and retention helpers are not enforced around every processing path. The database folder is a schema draft, not a live database integration.
- The repository website folder contains a static page, not an interactive hiring system. It is distinct from the company official website.

Human oversight and data safety

All Day 63 profiles and responses are fictional. Any real employment decision must be made by an authorized human reviewer using job-related evidence. Candidate data should be processed only with appropriate consent and access controls. Review repository data and generated reports for personal information before sharing; use the separate Day 63 synthetic dataset for public demonstrations.

Skills demonstrated by the project

- Modular Python development and data-contract design across document parsing and scoring components.
- Pytest-based validation, edge-case analysis, test isolation, and repeatable synthetic simulations.
- Sentence Transformer-based semantic matching, embedding-cache measurement, and performance reporting.
- Explainable score composition, fairness review, consent and retention concepts, and clear communication of system limits.

Recommended next steps

Follow the phased AI roadmap: connect the parser and ATS through a tested service boundary; enforce security and retention controls; evaluate scoring against consented human-reviewed data; safely isolate any code execution; then consider a limited, approved internal pilot. Set measurable gates with stakeholders before each phase and expand only features that meet them.

Portfolio conclusion

ZECPATH AI demonstrates a broad set of modular hiring-support components and a validated synthetic workflow. Its strongest next milestone is integration and evidence-based evaluation, with human accountability, privacy, and safe operation treated as release requirements.